DATA STRUCTURES – IMPORTANT NOTES

Exam-oriented notes covering the major topics

1. Introduction to Data Structures

Meaning: A data structure is a way of organizing and storing data so that it can be accessed and modified efficiently.

Types: Primitive: int, float, char, boolean. **Non-primitive:** arrays, linked lists, stacks, queues, trees and graphs.

Linear vs Non-linear: Linear: elements are arranged sequentially, e.g., array, linked list, stack, queue. **Non-linear:** hierarchical/network relationship, e.g., tree and graph.

Important terms: ADT (Abstract Data Type) describes data and permitted operations without specifying implementation. Common operations are insertion, deletion, searching, sorting and traversal.

2. Arrays

Definition: An array stores elements of the same type in contiguous memory locations and is accessed using an index.

Operations: Traversal, insertion, deletion, searching and updating.

Advantages: Fast random access using index; simple implementation.

Disadvantages: Usually fixed size; insertion and deletion in the middle can be costly.

Complexity: Access: O(1). Search: O(n) normally, O(log n) for binary search on a sorted array. Insertion/deletion: O(n) in the general case.

3. Linked Lists

Definition: A linked list consists of nodes. Each node contains data and one or more links to other nodes.

Types: Singly linked list, doubly linked list and circular linked list.

Singly list: Each node has data and a next pointer. The last node points to NULL.

Doubly list: Each node has previous and next pointers, allowing traversal in both directions.

Circular list: The last node links back to the first node.

Advantages: Dynamic size; insertion/deletion can be efficient when the required node position is known.

Disadvantages: Extra memory for links; no direct/random access.

Complexity: Access/search: O(n). Insertion/deletion at a known linked position: O(1).

4. Stack

Definition: A stack is a linear data structure that follows LIFO (Last In, First Out).

Operations: Push: insert. **Pop:** remove top element. **Peek/Top:** view top element. **isEmpty:** check whether empty.

Applications: Function calls/recursion, undo operations, expression conversion/evaluation and backtracking.

Errors: Overflow occurs when pushing into a full stack; underflow occurs when popping from an empty stack.

Implementation: Can be implemented using arrays or linked lists.

5. Queue

Definition: A queue is a linear data structure that follows FIFO (First In, First Out).

Operations: Enqueue: insert at rear. **Dequeue:** remove from front. **Front/Peek:** inspect the first element.

Types: Simple queue, circular queue, priority queue and deque.

Circular queue: The last position connects logically to the first, making better use of array space.

Applications: CPU scheduling, printer queues, buffering and breadth-first search.

Implementation: Arrays or linked lists.

6. Searching

Linear Search: Checks elements one by one. Works on sorted or unsorted data. Time: O(n).

Binary Search: Works on sorted data. Repeatedly divides the search interval into two halves. Time: O(log n).

Key point: Binary search is faster for large sorted arrays, while linear search is simpler and works without sorting.

7. Sorting

Bubble Sort: Repeatedly compares adjacent elements and swaps them if they are in the wrong order. Worst time: $O(n^2)$.

Selection Sort: Selects the minimum/maximum element and places it in the correct position. Time: $O(n^2)$.

Insertion Sort: Builds a sorted portion by inserting each new element into its proper position. Worst: $O(n^2)$; good for small/nearly sorted data.

Merge Sort: Divide-and-conquer algorithm. Divides the array, sorts halves and merges them. Time: O(n log n); extra space: O(n).

Quick Sort: Chooses a pivot and partitions elements around it. Average: O(n log n); worst: $O(n^2)$.

Exam point: Know the basic idea, algorithm steps, complexity and whether the sort is stable/in-place.

8. Trees

Definition: A tree is a non-linear hierarchical data structure consisting of nodes connected by edges.

Terms: Root, parent, child, sibling, leaf, internal node, degree, level, height and subtree.

Binary Tree: Each node has at most two children: left and right.

Binary Search Tree (BST): For every node, keys in the left subtree are smaller and keys in the right subtree are larger, according to the chosen duplicate policy.

BST operations: Search, insertion and deletion. Average time can be O(log n), but worst case can become O(n) if the tree becomes skewed.

9. Tree Traversals

Preorder: Root → Left → Right. Useful for copying/serializing tree structures.

Inorder: Left → Root → Right. In a BST, it gives keys in sorted order.

Postorder: Left → Right → Root. Useful when children must be processed before the parent.

Level Order: Visits nodes level by level, generally using a queue.

10. Balanced Search Trees

AVL Tree: A self-balancing BST in which the balance factor of every node is -1, 0 or +1.

Balance Factor: Height(left subtree) – Height(right subtree).

Rotations: LL → right rotation; RR → left rotation; LR → left rotation followed by right rotation; RL → right rotation followed by left rotation.

Complexity: Search, insertion and deletion are O(log n) because the height remains logarithmic.

Importance: Prevents a BST from becoming highly skewed and maintains efficient operations.

11. Heaps and Priority Queues

Heap: A complete binary tree satisfying the heap property.

Max Heap: Parent is greater than or equal to its children.

Min Heap: Parent is less than or equal to its children.

Operations: Insert and delete-root: O(log n); peek root: O(1); build heap: O(n).

Applications: Priority queues and heap sort.

12. Hashing

Definition: Hashing maps a key to an index using a hash function for fast average-case lookup.

Collision: Occurs when different keys map to the same index.

Collision handling: Chaining: store multiple entries in a bucket/list. **Open addressing:** find another empty position using probing.

Probing: Linear probing, quadratic probing and double hashing.

Complexity: Average search/insert/delete can be O(1); worst case can be O(n).

13. Graphs

Definition: A graph consists of vertices (nodes) and edges connecting them.

Types: Directed/undirected, weighted/unweighted, connected/disconnected, cyclic/acyclic.

Representation: Adjacency matrix: simple and fast edge lookup, uses $O(V^2)$ space. **Adjacency list:** space-efficient for sparse graphs, uses O(V+E) space.

BFS: Breadth-First Search explores level by level using a queue. Time: O(V+E) with adjacency lists.

DFS: Depth-First Search explores as far as possible before backtracking, using recursion or a stack. Time: O(V+E) with adjacency lists.

14. Important Graph Algorithms

Dijkstra: Finds shortest paths from a source when edge weights are non-negative.

Prim: Builds a minimum spanning tree by repeatedly choosing a minimum-weight edge that expands the tree.

Kruskal: Builds a minimum spanning tree by selecting edges in increasing weight order while avoiding cycles, commonly using Disjoint Set Union.

Topological Sort: Linear ordering of vertices in a directed acyclic graph (DAG) such that every directed edge u→v places u before v.

15. Recursion

Definition: Recursion is a technique where a function calls itself to solve smaller instances of a problem.

Parts: Base case: stops recursion. **Recursive case:** reduces the problem toward the base case.

Applications: Tree traversal, divide-and-conquer algorithms, DFS and backtracking.

Limitation: Deep recursion can consume significant call-stack memory.

16. Important Complexity Terms

Big-O: Describes an upper bound on growth of running time or space.

Common orders: O(1) constant, O(log n) logarithmic, O(n) linear, O(n log n), $O(n^2)$ quadratic.

Rule: For exams, remember that smaller growth rates are generally more efficient for large input sizes.

17. Frequently Asked Exam Questions

2-mark questions: Define data structure and ADT; differentiate stack and queue; define linked list; define tree and BST; define AVL tree; define BFS and DFS; define hashing and collision.

Long questions: Explain linked-list operations; implement stack and queue; explain circular queue; explain searching and sorting algorithms with complexity; explain tree traversals; construct/rotate an AVL tree; explain graph representations and BFS/DFS; explain hashing and collision resolution.

Quick Revision – Complexity Table

Topic	Typical / Important Time
Array access	O(1)
Linear search	O(n)
Binary search	O(log n)
Linked-list search/access	O(n)
Stack push/pop	O(1)
Queue enqueue/dequeue	O(1)
Bubble / Selection / Insertion (worst)	$O(n^2)$
Merge sort	O(n log n)
Quick sort (average / worst)	O(n log n) / $O(n^2)$
BST search (average / worst)	O(log n) / O(n)
AVL search/insert/delete	O(log n)
Heap insert/delete	O(log n)
Hashing (average / worst)	O(1) / O(n)
BFS / DFS with adjacency list	O(V+E)

Study tip: For each data structure, prepare definition, types, operations, advantages/disadvantages, applications and time complexity. For algorithms, learn the steps and one small example.